

```

import pandas as pd
import numpy as np
import cv2
import timm
import torch
import torch.nn as nn
import albumentations as A
import pytorch_lightning as pl
import matplotlib.pyplot as plt

from torch.utils.data import Dataset, DataLoader
from albumentations.core.composition import Compose, OneOf
from albumentations.augmentations.transforms import CLAHE, GaussNoise,
ISONoise
from albumentations.pytorch import ToTensorV2

from pytorch_lightning import Trainer, seed_everything
from pytorch_lightning import Callback
from pytorch_lightning.loggers import CSVLogger
from pytorch_lightning.callbacks import ModelCheckpoint, EarlyStopping

```

```

from sklearn.model_selection import StratifiedKFold

```

```

print(f"PyTorch Lightning version: {pl.__version__}")

```

PyTorch Lightning version: 1.2.0

Config

```

class CFG:
    seed = 42
    model_name = 'resnet50'
    pretrained = True
    img_size = 512
    num_classes = 12
    lr = 5e-4
    min_lr = 1e-6
    t_max = 20
    num_epochs = 20
    batch_size = 32
    accum = 1
    precision = 16
    n_fold = 5
    device = torch.device('cuda' if torch.cuda.is_available() else
'cpu')

```

```

PATH = "plant_data/"

```

```

TRAIN_DIR = "plant_data/resized-plant2021/img_sz_512/"

```

```

TEST_DIR = PATH + 'test_images/'

```

```

seed_everything(CFG.seed)

```

Global seed set to 42

42

```
df_all = pd.read_csv(PATH + "train.csv")
df_all.head()
```

	image	labels
0	800113bb65efe69e.jpg	healthy
1	8002cb321f8bfcdcf.jpg	scab frog_eye_leaf_spot complex
2	80070f7fb5e2ccaa.jpg	scab
3	80077517781fb94f.jpg	scab
4	800cbf0ff87721f8.jpg	complex

Label Encoding

```
labels = list(df_all['labels'].value_counts().keys())
labels_dict = dict(zip(labels, range(12)))
df_all = df_all.replace({"labels": labels_dict})
df_all.head()
```

	image	labels
0	800113bb65efe69e.jpg	1
1	8002cb321f8bfcdcf.jpg	7
2	80070f7fb5e2ccaa.jpg	0
3	80077517781fb94f.jpg	0
4	800cbf0ff87721f8.jpg	4

Split Train Data

```
sfk = StratifiedKFold(CFG.n_fold)
for train_idx, valid_idx in sfk.split(df_all['image'],
df_all['labels']):
    df_train = df_all.iloc[train_idx]
    df_valid = df_all.iloc[valid_idx]
    break
```

```
print(f"train size: {len(df_train)}")
print(f"valid size: {len(df_valid)}")
```

train size: 14905
valid size: 3727

Define Dataset

```
class PlantDataset(Dataset):
    def __init__(self, df, transform=None):
        self.image_id = df['image'].values
        self.labels = df['labels'].values
        self.transform = transform

    def __len__(self):
        return len(self.labels)
```

```

def __getitem__(self, idx):
    image_id = self.image_id[idx]
    label = self.labels[idx]

    image_path = TRAIN_DIR + image_id
    image = cv2.imread(image_path)
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    augmented = self.transform(image=image)
    image = augmented['image']
    return {'image': image, 'target': label}

def get_transform(phase: str):
    if phase == 'train':
        return Compose([
            A.RandomResizedCrop(height=CFG.img_size,
width=CFG.img_size),
            A.HorizontalFlip(p=0.5),
            A.ShiftScaleRotate(p=0.5),
            A.RandomBrightnessContrast(p=0.5),
            A.Normalize(),
            ToTensorV2(),
        ])
    else:
        return Compose([
            A.Resize(height=CFG.img_size, width=CFG.img_size),
            A.Normalize(),
            ToTensorV2(),
        ])

train_dataset = PlantDataset(df_train, get_transform('train'))
valid_dataset = PlantDataset(df_valid, get_transform('valid'))

train_loader = DataLoader(train_dataset, batch_size=CFG.batch_size,
shuffle=True, num_workers=2, pin_memory=True, drop_last=True)
valid_loader = DataLoader(valid_dataset, batch_size=CFG.batch_size,
shuffle=False, num_workers=2)

```

Define Model

```

class CustomResNet(nn.Module):
    def __init__(self, model_name='resnet18', pretrained=False):
        super().__init__()
        self.model = timm.create_model(model_name,
pretrained=pretrained)
        in_features = self.model.get_classifier().in_features
        self.model.fc = nn.Linear(in_features, CFG.num_classes)

    def forward(self, x):
        x = self.model(x)
        return x

```

```

class Plant(pl.LightningModule):
    def __init__(self, model):
        super(Plant, self).__init__()
        self.model = model
        self.metric = pl.metrics.F1(num_classes=CFG.num_classes)
        self.criterion = nn.CrossEntropyLoss()
        self.lr = CFG.lr

    def forward(self, x, *args, **kwargs):
        return self.model(x)

    def configure_optimizers(self):
        self.optimizer = torch.optim.Adam(self.model.parameters(),
lr=self.lr)
        self.scheduler =
torch.optim.lr_scheduler.CosineAnnealingLR(self.optimizer,
T_max=CFG.t_max, eta_min=CFG.min_lr)

        return {'optimizer': self.optimizer, 'lr_scheduler':
self.scheduler}

    def training_step(self, batch, batch_idx):
        image = batch['image']
        target = batch['target']
        output = self.model(image)
        loss = self.criterion(output, target)
        score = self.metric(output.argmax(1), target)
        logs = {'train_loss': loss, 'train_f1': score, 'lr':
self.optimizer.param_groups[0]['lr']}
        self.log_dict(
            logs,
            on_step=False, on_epoch=True, prog_bar=True, logger=True
        )
        return loss

    def validation_step(self, batch, batch_idx):
        image = batch['image']
        target = batch['target']
        output = self.model(image)
        loss = self.criterion(output, target)
        score = self.metric(output.argmax(1), target)
        logs = {'valid_loss': loss, 'valid_f1': score}
        self.log_dict(
            logs,
            on_step=False, on_epoch=True, prog_bar=True, logger=True
        )
        return loss

```

```

model = CustomResNet(model_name=CFG.model_name,
pretrained=CFG.pretrained)
plant_model = Plant(model)

logger = CSVLogger(save_dir='logs/', name=CFG.model_name)
logger.log_hyperparams(CFG.__dict__)
checkpoint_callback = ModelCheckpoint(monitor='valid_loss',
                                     save_top_k=1,
                                     save_last=True,
                                     save_weights_only=True,

filename='checkpoint/{epoch:02d}-{valid_loss:.4f}-{valid_f1:.4f}',
                                     verbose=False,
                                     mode='min')

trainer = Trainer(
    max_epochs=CFG.num_epochs,
    accumulate_grad_batches=CFG.accum,
    checkpoint_callback=checkpoint_callback,
    logger=logger,
    weights_summary='top',
)

```

```

GPU available: False, used: False
TPU available: None, using: 0 TPU cores

```

Training

```

trainer.fit(plant_model, train_dataloader=train_loader,
val_dataloaders=valid_loader)

```

Results

```

metrics = pd.read_csv(f'{trainer.logger.log_dir}/metrics.csv')

train_acc = metrics['train_f1'].dropna().reset_index(drop=True)
valid_acc = metrics['valid_f1'].dropna().reset_index(drop=True)

fig = plt.figure(figsize=(7, 6))
plt.grid(True)
plt.plot(train_acc, color="r", marker="o", label='train/f1')
plt.plot(valid_acc, color="b", marker="x", label='valid/f1')
plt.ylabel('F1', fontsize=24)
plt.xlabel('Epoch', fontsize=24)
plt.legend(loc='lower right', fontsize=18)
plt.savefig(f'{trainer.logger.log_dir}/f1.png')

train_loss = metrics['train_loss'].dropna().reset_index(drop=True)
valid_loss = metrics['valid_loss'].dropna().reset_index(drop=True)

fig = plt.figure(figsize=(7, 6))
plt.grid(True)

```

```
plt.plot(train_loss, color="r", marker="o", label='train/loss')
plt.plot(valid_loss, color="b", marker="x", label='valid/loss')
plt.ylabel('Loss', fontsize=24)
plt.xlabel('Epoch', fontsize=24)
plt.legend(loc='upper right', fontsize=18)
plt.savefig(f'{trainer.logger.log_dir}/loss.png')\
```

```
lr = metrics['lr'].dropna().reset_index(drop=True)
```

```
fig = plt.figure(figsize=(7, 6))
plt.grid(True)
plt.plot(lr, color="g", marker="o", label='learning rate')
plt.ylabel('LR', fontsize=24)
plt.xlabel('Epoch', fontsize=24)
plt.legend(loc='upper right', fontsize=18)
plt.savefig(f'{trainer.logger.log_dir}/lr.png')
```